Assignment 2: From Trees to Neural Networks

Bank Marketing Dataset Report

This report compares Gradient Boosted Decision Trees (GBDT, implemented with XGBoost) and a Multi-Layer Perceptron (MLP) on the Bank Marketing dataset. The dataset contains 45,211 rows with a positive class rate of 11.7%. The goal is to predict whether a customer subscribes to a term deposit while keeping preprocessing strictly training-only to avoid leakage.

The experiment follows the assignment rubric directly: train/validation/test splitting, categorical encoding, MLP feature scaling, GBDT sweeps over learning rate and n_estimators, MLP sweeps over architecture and optimization settings, and a final side-by-side comparison on Accuracy, Precision, Recall, F1-score, AUC-PR, and training time.

This dataset is a good stress test for both families because it combines moderate class imbalance, mixed feature types, and nonlinear interactions between campaign history and customer attributes.

Split	Rows	Positive rate
Train	31,647	11.7%
Validation	6,782	11.7%
Test	6,782	11.7%

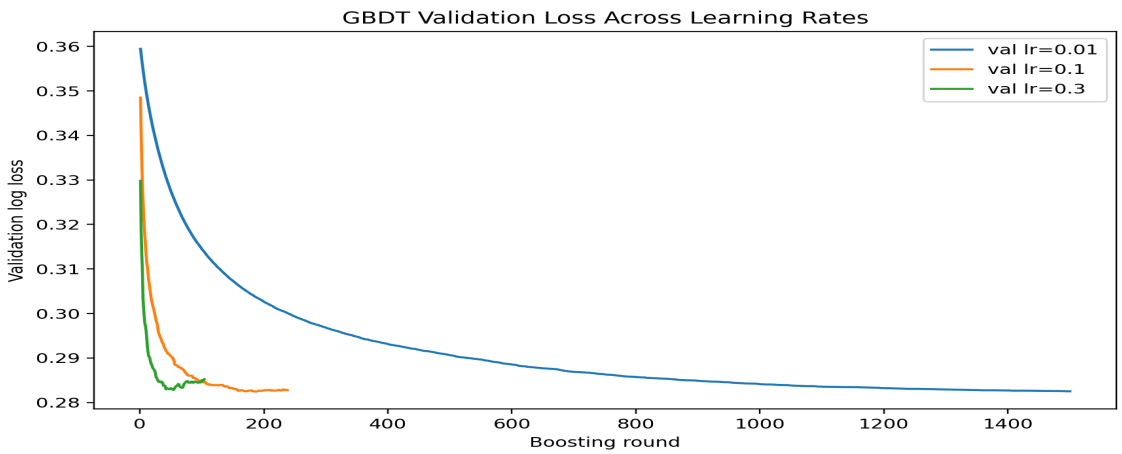
Methods

Missing-value handling converted `unknown` placeholders in `job`, `education`, `contact`, and `poutcome` into true missing values. Numeric variables used median imputation, while categorical variables used constant-value imputation followed by one-hot encoding. All preprocessing objects were fit on the training split only and then reused unchanged on validation and test, which avoids data leakage.

Feature engineering added `age_group`, `balance_log_abs`, `contacts_total`, `was_previously_contacted`, and `pdays_clean`. These transformations were intended to reduce skew, separate first-time from repeat-contact cases, and preserve useful prior-campaign information. The raw `duration` field was removed because it is only known after the marketing call and would leak future information.

StandardScaler was applied only to the MLP branch because neural networks are sensitive to relative feature magnitudes. Trees do not need that step because they learn threshold splits rather than weight vectors in a continuous feature space.

Learning rate	Best iteration	Val F1	Val AUC-PR	Train time (s)
0.01	1500	0.345	0.463	12.231
0.1	188	0.353	0.462	2.757
0.3	54	0.368	0.461	0.747



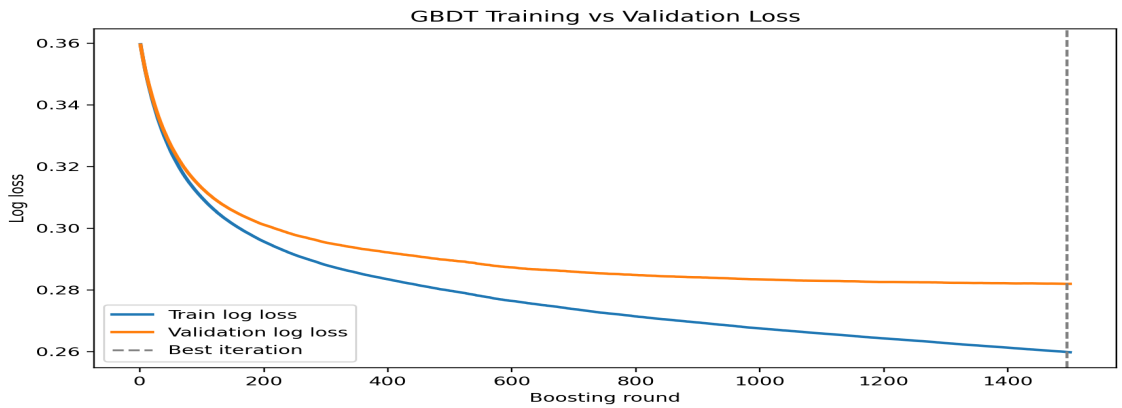
n_estimators	Effective trees	Val F1	Val AUC-PR	Train time (s)
200	200	0.253	0.421	1.470
600	600	0.307	0.449	3.920
1500	1500	0.345	0.463	9.411

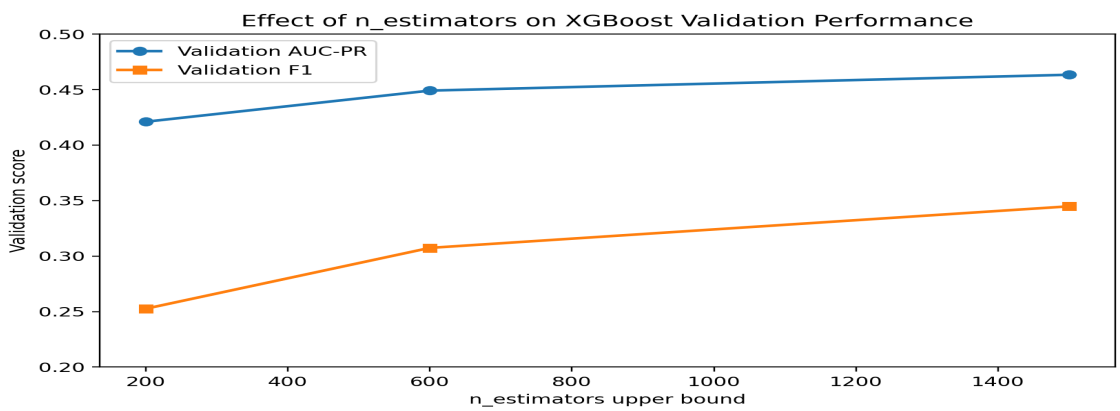
GBDT Results

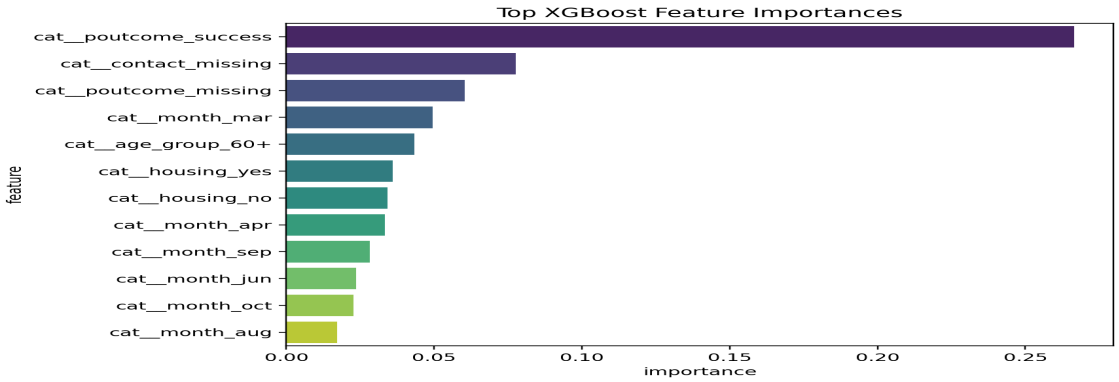
The best tree model used learning_rate=0.01, max_depth=5, subsample=0.9, reg_alpha=1.0, and reg_lambda=10.0. The main pattern is that slower, more gradual boosting improved ranking quality, while faster settings reduced training time but were more likely to trade away AUC-PR.

Concretely, learning_rate=0.01 produced the best validation AUC-PR, while learning_rate=0.3 was the fastest option at 0.747 seconds. The n_estimators sweep shows the same bias-reduction story: validation AUC-PR increased from 0.421 at 200 trees to 0.463 at 1500 trees, so the smaller ensembles were still leaving useful structure unexplained.

Metric	Validation	Test
Accuracy	0.896	0.894
Precision	0.650	0.630
Recall	0.241	0.226
F1	0.351	0.332
AUC-PR	0.466	0.456
Train time (s)	10.016	10.016





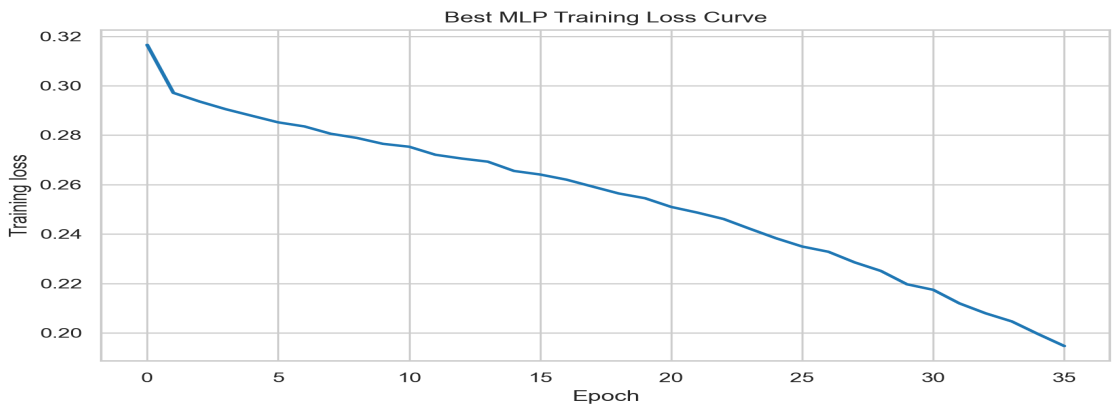

Top XGBoost Feature Importances
cat__poutcome_success
cat__contact_missing
cat__poutcome_missing
cat__month_mar
cat__age_group_60+
cat__housing_yes
cat__housing_no
cat__month_apr
cat__month_sep
cat__month_jun
cat__month_oct
cat__month_aug
feature
0.00
0.05
0.10
0.15
0.20
0.25
importance

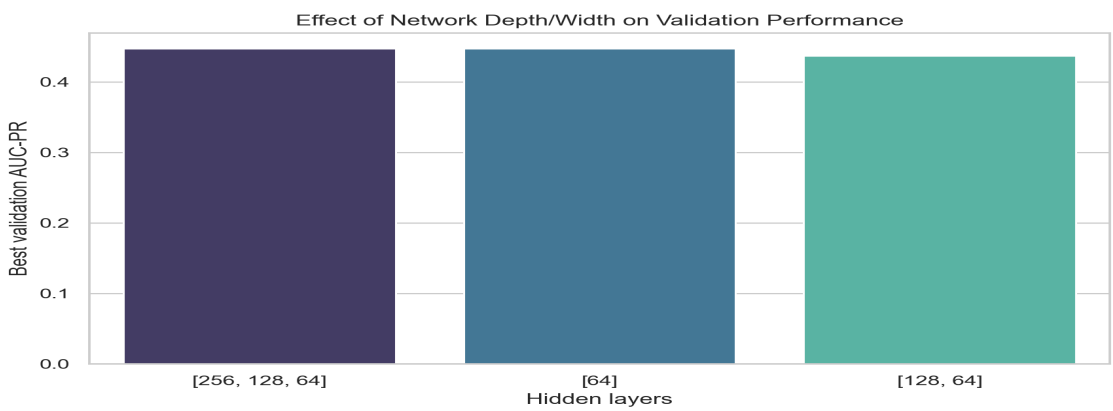
MLP Results

The strongest MLP used hidden layers (256, 128, 64), tanh activation, and learning_rate_init=0.001. That combination favored stable optimization over aggressive updates, which matters when one-hot encoded tabular inputs produce a relatively sparse, uneven feature space.

The max_iter sweep is informative even though the scores are identical. Every run stopped after 36 epochs, so the effective limit came from early stopping rather than the iteration cap itself. That suggests the remaining performance gap was not due to an insufficient number of epochs.

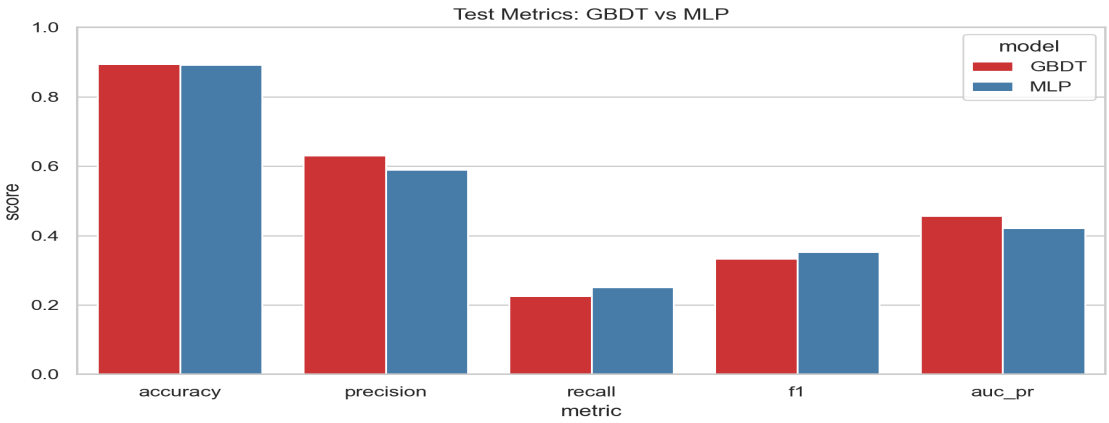
Metric	Validation	Test
Accuracy	0.895	0.892
Precision	0.614	0.589
Recall	0.279	0.251
F1	0.383	0.352
AUC-PR	0.447	0.421
Train time (s)	77.833	77.833





GBDT vs MLP Comparison

Model	Accuracy	Precision	Recall	F1	AUC-PR	Train time (s)
GBDT	0.894	0.630	0.226	0.332	0.456	10.016
MLP	0.892	0.589	0.251	0.352	0.421	77.833



XGBoost achieved test AUC-PR 0.456 versus 0.421 for the MLP, while the MLP slightly exceeded XGBoost on F1 (0.352 vs 0.332). This split indicates that the tree model ranked examples more reliably across thresholds, even though the neural network found a slightly better default operating point at the fixed 0.5 cutoff.

Overall, GBDT remains the stronger default here. It trained about 7.8 times faster, required less sensitive preprocessing, and offered direct feature-importance analysis. The MLP still has value as a flexible baseline, especially if the project later expands toward richer neural architectures.

Discussion, Limitations, and AI Disclosure

Several limitations matter when interpreting the results. First, the dataset is class-imbalanced, so accuracy alone would overstate quality; AUC-PR and F1 are more informative for the positive class. Second, even after removing `duration`, some campaign variables may still reflect operational context that shifts over time. Third, this is a focused model-comparison study rather than an exhaustive leaderboard search.

The bias-variance picture also matches the course objective. Boosting improved as capacity grew gradually through more trees and smaller learning steps, while the MLP already had enough representational power and mainly depended on whether optimization could find a stable solution before plateauing. Early stopping therefore helped both families, but for different reasons.

AI Tool Disclosure: OpenAI Codex was used to help structure the notebook workflow, automate the experiment pipeline, and draft report prose. The model choices, leakage handling decision for `duration`, experiment execution, and generated artifacts in this workspace were all assembled within this session.